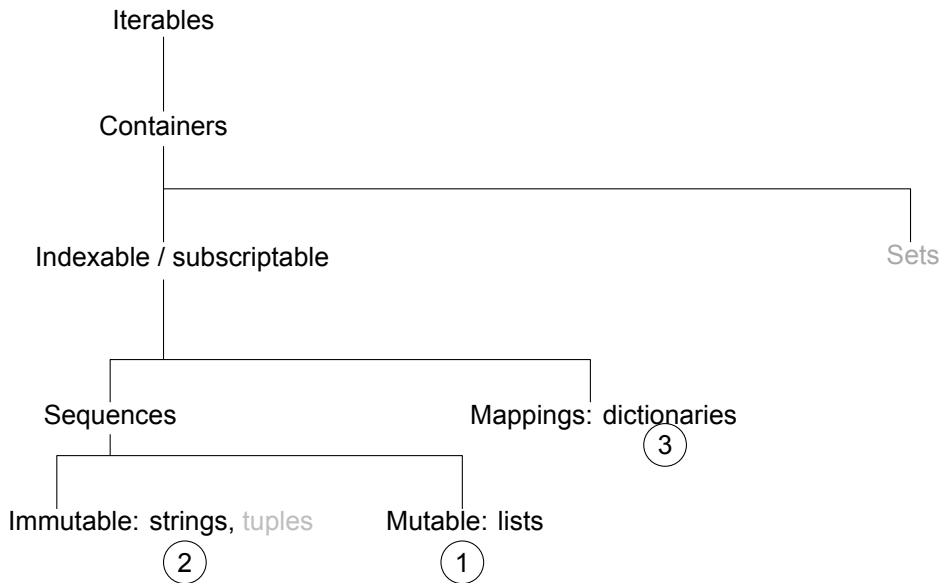


Python: container types (lists, strings, dictionaries)

Intro. to Computing

<https://mandarm.github.io/computing.html>

Container types



Container types

■ Lists: **mutable** sequence

- elements accessed using *indices*, and **can be changed**
- creating lists

```
l = [ 11, "y", 7.4 ]  
l = []           # empty list  
l = list(range(0,100,10)) # generally, list(iterable)  
numbers = [10, 16, -2, 22, -98]
```

LATER: using list comprehension

■ Strings (str) : **immutable** sequence

- elements accessed using *indices*, but **cannot be changed**

■ Dictionaries { 1:11, 2:"y", 3:7.4, 3.14:"pi" }

- elements accessed using *keys* (must be distinct)

■ Tuples (11, "y", 7.4)

■ Sets { 11, "y", 7.4 } — *not “subscriptable” (indexable)*

Lists: operations

(Applicable to immutable sequences also)

Operation	Result
<code>x in s</code>	True if an item of <code>s</code> is equal to <code>x</code> , else False ^{***}
<code>x not in s</code>	False if an item of <code>s</code> is equal to <code>x</code> , else True ^{***}
<code>s + t</code>	concatenation of <code>s</code> and <code>t</code> , does not change <code>s</code> , <code>t</code>
<code>s * n</code> or <code>n * s</code>	equivalent to concatenating <code>s</code> <code>n</code> times ^{††}
<code>len(s)</code>	length of <code>s</code>

^{***} For strings, `x` can also be a substring, e.g.,
`"gg" in "eggs" "ana" in "banana"`

^{††} **LATER:** may lead to unexpected behaviour

Exercise: Try using a negative value for `n`.

Lists: indexing and slicing

Indexing

negative indexes

s =

positive indexes

-n	-(n+1)	...	-2	-1
[10,	20,	...	40,	50,]
0	1	...	n-2	n-1

NOTE:
 $-0 \equiv 0$

Slicing: `s[start:end:step]`

- start: starting index, **default** 0, included in slice
- end: ending index, **default** `len(s)`, excluded from slice
- step: skip step elements
(negative step $\Rightarrow$ reverse direction, `step == 0` not permitted)
- Replace out of range values by 0 or `len(s)`, as appropriate.
- If `start >= end`, the slice is empty.

Note the difference with `start`, `stop` for `range()`.

Lists: more operations

(Applicable to mutable sequences only)

Operation	Result
<code>s[i] = x</code>	item <code>i</code> of <code>s</code> is replaced by <code>x</code>
<code>del s[i]</code>	removes item <code>i</code> of <code>s</code>
<code>s[i:j] = t</code>	slice of <code>s</code> from <code>i</code> to <code>j</code> is replaced by the contents of the <code>t</code>
<code>del s[i:j:k]</code>	removes the elements of <code>s[i:j:k]</code> from the list, <code>:k</code> optional same as <code>s[i:j:k] = []</code>
<code>s[i:j:k] = t</code>	elements of <code>s[i:j:k]</code> are replaced by those of <code>t</code> <code>:k</code> optional, but if not 1, <code>s[i:j:k]</code> and <code>t</code> must have same length
<code>s += t</code>	extends <code>s</code> with contents of <code>t</code>
<code>s *= n</code>	updates <code>s</code> with its contents repeated <code>n</code> times

Lists: some useful functions I

(Applicable to immutable sequences also)

- `s.count(val)`: number of occurrences of `val` in sequence `s`

- `s.index(val)`

 - `s.index(value, start)`

 - `s.index(value, start, stop)`

index of the **first** occurrence of `value` in sequence `s`, but **ERROR** if it does not occur

LATER: exceptions (errors) and how to handle them

Lists: some useful functions II

(Applicable to mutable sequences only)

- `s.append(value)`
- `s.pop(index)`: index optional, **default** is -1
get the item at index and remove it from the sequence.
- `s.remove(value)`: remove the first item that matches value, but **ERROR** if it does not occur
- `s.extend(sequence)`
- `s.insert(index, value)`
- `s.reverse()`, `reversed(s)`: **NOTE the difference!**

Lists: more useful functions

If the elements can be compared

- `min(l)` smallest item of `l`
- `max(s)` largest item of `s`
- `sort()`, `sorted()`
- `sum()` for numeric types

```
numbers = [10, 16, -2, 22, -98]
sorted(numbers)      # [-98, -2, 10, 16, 22]
reversed(numbers)    # <list_reverseiterator object at ...>
numbers.sort()       # changes IN PLACE
numbers.reverse()    # changes IN PLACE
```

Note the difference between `sorted()` (returns a list) and `reversed()` (returns an iterable).